Jordan Pauzie & Julian Savini
CS: 151-2 Special Topics in Data Infrastructure and Systems
Professor Lillethun & Professor Sundar
10/30/25

Project 2 Final Writeup

Design Overview [Beta]:

Our initial plan of action was to implement the projection algorithm for render() and then optimize smaller portions of serial code such as invariant color calculations and symmetry of gravitational force. The high-level idea of our projection algorithm as to sort all the spheres by distance to the eye, filter out all the spheres whose centers weren't within the scope of the projection plane (to reduce the number of potential calculations), construct bounding boxes around the 2D circular images of the spheres (because from the perspective of the eye and projection plane, every sphere appears to be a circle), and then attempt to map points from the projection plane to the points on the sphere (within the restricted bounds). We would then maintain a 1D "filled" array which keeps track as to whether a particular point on the projection plane had already been colored or not; this would allow us to avoid iterating over already-filled pixels and save computation time. However in our initial implementation of the projection algorithm, there were several issues. Firstly, spheres on the edges of the plane were being omitted and secondly, their 2D circular images were being cut off due to the bounds being too tight (and not centered for some reason). We fixed the edge-sphere issue by computing bounding boxes for every sphere and then bounded those coordinates by the coordinates of the projection plane, and the tight bound issue by doubling the size of the bounds (as a temporary fix). In doing so, our new design was able to achieve tier 24 on ./tiers.

We then attempted to handle precomputing color invariants by augmenting the sphere struct but ran into the issue of the render() function declaring all spheres passed in as static; this meant that I wouldn't be able to store invariant data within the sphere, and proved to be a major stumbling block. Instead, we decided to obtain a speed-up by simply parallelizing render() using OpenCilk. Two areas of logic were parallelized: the zero-initialization of the filled[] array used to keep track of colored pixels, and the double for-loop which handled coloring pixels. Since the actual computation of red, green, blue relied on information unique to that particular pair of x and y coordinates, each pixel-coloring was functional independently of one another. This meant that parallelizing the outer for-loop was completely safe from data dependencies, and performance was improved significantly. These optimizations were further built upon by our team's optimization of simulate.c where we focused on exploiting symmetry.

Exploiting symmetry involves understanding Newton's third law. The same pull i has on j, is equal to j's pull i on, just as a negative value. Instead of computing the forces twice, we compute each pair of forces one time. If there are n spheres, whereas it would take n^2 calculations, it is now n^2 / 2. All pairwise forces are computed into a cache, and then they are stacked in the same order as the original algorithm, so it can pass correctness tests. For situations where spheres have very large masses, we decided to use the original algorithm. This is because although the gifs look accurate, we assume there are floating point miscalculations, which are thus failing correctness. We parallelized update_accelerations_libstudent() by adding cilk_for to

the accumulation loop. This loop only reads from force_cache and writes to spheres[i + n_spheres].accel, so there are no data races.

Design Overview [Improvements]:
This section will outline what improvements were made to our code, during the final submission week.

When considering what parts of the code could be further optimized, it was important to consider what functions or portions of logic were considered “unskippable” in the eyes of our program flow. With this in mind, a very obvious candidate for optimization was the sort() function. Whenever a frame is to be rendered, render() always calls sort() to return the sorted list of spheres which are then rendered in order of closest to farthest. The base sort() function was implemented as an insertion sort which subsequently returned the sorted array of spheres. While efficient for smaller quantities of spheres, this sorting algorithm struggles with larger cases and is a bad candidate for parallelization. Thus we rewrote sort() to be a coarsened, parallelized merge sort that switched back to insertion sort for smaller sizes of *n*. Merge sort as an algorithm is recursive in nature, operating by splitting each subarray into two smaller subarrays, sorting them, and then merging after all lower level calls have been completed. Because of the subarrays all maintaining independence from each other, it makes sense that the left and right recursive calls can be performed concurrently through the use of Cilk_spawn and Cilk_sync.

Another modification was the elimination of redundant computations with pow() and qsize(). qsize() is used to compute the distance, and pow() is used to cube said distances. pow() is slow as it handles fractions and negative powers. Thus, these were all replaced with multiplication statements. qsize() was redundantly calculated. Dist and distance store i_minus_j and i_to_j, and are called throughout the update_accelerations_libstudent function. The same is done in the check_for_collision function, where moveSpeed is stored and then used to calculate timeToCollision. Although this a less fancy improvement, it did help us reach an additional tier.

By now our code was achieving a consistent tier 50, and in our pursuit for tier 60+ we began searching for micro-optimizations that could help bump up one tier at a time (this was before we implemented the final optimization described in the final paragraph). Thus various miscellaneous optimizations were implemented including manual inlining, invariant precomputations and logic concision. In general, compilers nowadays are quite good at replacing function calls with their inlined versions and performing other assembly-level optimizations. However this isn’t always necessarily the case, and we found that manually inlining some function calls (especially for vector computations) and flattening single-use variables provided a small upside. Earlier in the beta stage, we had an issue with our bounding box (used to determine what pixels on the spheres mapped to the projection plane) being off-center from our spheres. In order to fix this, we doubled the size of our bounding boxes which fixed the problem but resulted in wasteful computation. While we were unable to fix such an issue in the final submission, the bounds padding was shaved down to 1.5x instead of 2x. Finally, we noticed that for every (x, y) coordinate in the projection plane being iterated within render(), a ray was being instantiated at every loop iteration to perform the pixel coloring

process. Instead, we decided to precomputate every possible ray within the (x, y) range into an array using a parallelized for-loop which then allowed us to avoid continuously recalculating the ray during the coloring loop.

The final major change was to parallelize the do_timestep_libstudent function. The objective of do_timestep_libstudent is to call collision checking between each pair of spheres to see if they will collide, in the next timestep. Originally we thought there would be a determinacy race, but since checking on collisions does not depend on the previous check before it, this is a good function to parallelize. A list, collision_times, was used to store the potential collision times of the sphere pairs. Thus, regardless of the order in which these collision times are written, they have their own spot within the list to be written. Cilk_for was applied to the outermost loop, in order to encourage parallelization. After the parallelization is done, the result is a list of collision times for every sphere pair. After this step, the array is iterated through to find the minimum collision time, which identifies which two spheres are colliding and when it will occur. At that collision time, the physics have changed, and it will have to be recalculated.

Additional Information & Acknowledgements:

- Julian spent quite some time on a method to take the center of the sphere and its radius, get coordinate pixels for the 8 corners of a square, and then broadcast those points onto the 2D plane. This ultimately did not work, but it is worth sharing a failed implementation.
- AI tools were used for idea generation and debugging, adhering to the AI policy.

Tier 2 Performance Tracking:

Version	**Tier**	**Changes (if any)**
Benchmark	30	
Original	0	
Version 1	24	Replaced ray tracing algorithm with projection algorithm
Version 2	32	Exploited symmetry using Cilk
Version 3	47	Parallelized render() function
Version 4	49	MergeSort implementation
Version 5	50	Removed redundant qsize and pow operations
Version 6	52	Manual inlining + loop fixes
Version 7	64	Parallelize do_timestep

Beta Project Log:

Important Note: We met and worked together on all parts of the project.

#	Date	Duration	Description
1	October 15	2	Carefully reading instructions, understanding problem space, reading files
2	October 15	1	Implementation of 8 corners to capture planet, failed implementation
3	October 15	1	Partial implementation of 3D projection algorithm
4	October 17	6	Debugging 3D projection algorithm
5	October 20	2	Exploiting symmetry, nearly correct implementation
6	October 20	2	Color invariance
7	October 22	.5	Looking current work, strategizing until stampede3 is up
8	October 22	1	Write up
9	October 22	3	Exploiting symmetry fixed, implemented standard calc for larger spheres
10	October 23	1	Parallelized acceleration function + merging code
11	October 23	1	Write up

FINAL Project Log:

Important Note: We met and worked together on all parts of the project.

#	Date	Duration	Description
1	October 27	3	Mergesort implementation
2	October 27	2	Trying to make earlier breaks [failed] + removed redundant pow and qsize operations
3	October 27	.5	Debugging, cleaning main branch
4	October 29	4	Parallelizing do_timestep, ray_t precomputation, manual inlining, and logic concision
5	October 30	.5	Code cleaning, documentation
6	October 30	1	Final write-up

Screenshots:

Below are a couple of snapshots showing our success in idev:

```
c454-092[skx](995)$ ./scripts/test_simulations
./simulations/100: PASS
./simulations/1000: PASS
./simulations/15: PASS
./simulations/150: PASS
./simulations/30: PASS
./simulations/300: PASS
./simulations/50: PASS
./simulations/500: PASS
./simulations/collision: PASS
./simulations/one_sphere: PASS
./simulations/rgb1: PASS
./simulations/rgb4: PASS
./simulations/static: PASS
./simulations/sun: PASS
```

```
c454-092[skx](994)$ ./bin/find-tier
REMINDER: This is only timing, NOT CORRECTNESS. Ensure you also run correctness tests
PASS (eoo!):    Tier 0 :       Ran 320x320     image with 800 bodies in 20ms (s: 8, r: 10, is: 0, ir: 0)
PASS (yeee!):   Tier 1 :       Ran 320x320     image with 832 bodies in 11ms (s: 3, r: 6, is: 0, ir: 0)
PASS (ayy!):    Tier 2 :       Ran 336x336     image with 865 bodies in 10ms (s: 3, r: 4, is: 0, ir: 0)
PASS (skrrt!):  Tier 3 :       Ran 352x352     image with 899 bodies in 12ms (s: 3, r: 6, is: 0, ir: 0)
PASS (yeee!):   Tier 4 :       Ran 368x368     image with 935 bodies in 14ms (s: 3, r: 8, is: 0, ir: 0)
PASS (woot!):   Tier 5 :       Ran 384x384     image with 973 bodies in 14ms (s: 4, r: 7, is: 0, ir: 0)
PASS (boyah!):  Tier 6 :       Ran 400x400     image with 1012 bodies in 16ms (s: 5, r: 9, is: 0, ir: 0)
PASS (skrrt!):  Tier 7 :       Ran 416x416     image with 1052 bodies in 16ms (s: 4, r: 9, is: 0, ir: 0)
PASS (skrrt!):  Tier 8 :       Ran 432x432     image with 1094 bodies in 18ms (s: 6, r: 10, is: 0, ir: 0)
PASS (yeee!):   Tier 9 :       Ran 448x448     image with 1138 bodies in 21ms (s: 6, r: 12, is: 0, ir: 0)
PASS (boyah!):  Tier 10 :      Ran 464x464     image with 1184 bodies in 21ms (s: 6, r: 12, is: 0, ir: 0)
PASS (yay!):    Tier 11 :      Ran 480x480     image with 1231 bodies in 22ms (s: 6, r: 13, is: 0, ir: 0)
PASS (eoo!):    Tier 12 :      Ran 512x512     image with 1280 bodies in 25ms (s: 7, r: 15, is: 0, ir: 0)
PASS (eoo!):    Tier 13 :      Ran 528x528     image with 1332 bodies in 26ms (s: 7, r: 16, is: 0, ir: 0)
PASS (ayy!):    Tier 14 :      Ran 544x544     image with 1385 bodies in 29ms (s: 9, r: 18, is: 0, ir: 0)
PASS (ayy!):    Tier 15 :      Ran 576x576     image with 1440 bodies in 29ms (s: 9, r: 18, is: 0, ir: 0)
PASS (yay!):    Tier 16 :      Ran 592x592     image with 1498 bodies in 33ms (s: 9, r: 22, is: 0, ir: 0)
PASS (ayy!):    Tier 17 :      Ran 608x608     image with 1558 bodies in 34ms (s: 9, r: 22, is: 0, ir: 0)
PASS (ayy!):    Tier 18 :      Ran 640x640     image with 1620 bodies in 37ms (s: 10, r: 24, is: 0, ir: 0)
PASS (eoo!):    Tier 19 :      Ran 672x672     image with 1685 bodies in 40ms (s: 12, r: 26, is: 0, ir: 0)
PASS (woot!):   Tier 20 :      Ran 688x688     image with 1752 bodies in 42ms (s: 12, r: 27, is: 0, ir: 0)
PASS (yay!):    Tier 21 :      Ran 720x720     image with 1823 bodies in 44ms (s: 13, r: 28, is: 0, ir: 0)
PASS (yeee!):   Tier 22 :      Ran 752x752     image with 1895 bodies in 51ms (s: 15, r: 34, is: 0, ir: 0)
PASS (eoo!):    Tier 23 :      Ran 784x784     image with 1971 bodies in 52ms (s: 15, r: 34, is: 0, ir: 0)
PASS (ayy!):    Tier 24 :      Ran 816x816     image with 2050 bodies in 61ms (s: 17, r: 40, is: 0, ir: 0)
PASS (yay!):    Tier 25 :      Ran 848x848     image with 2132 bodies in 62ms (s: 18, r: 41, is: 0, ir: 0)
PASS (yeee!):   Tier 26 :      Ran 880x880     image with 2217 bodies in 66ms (s: 19, r: 43, is: 0, ir: 0)
PASS (skrrt!):  Tier 27 :      Ran 912x912     image with 2306 bodies in 71ms (s: 21, r: 49, is: 0, ir: 0)
PASS (yay!):    Tier 28 :      Ran 944x944     image with 2398 bodies in 78ms (s: 22, r: 52, is: 0, ir: 0)
PASS (yay!):    Tier 29 :      Ran 992x992     image with 2494 bodies in 88ms (s: 24, r: 61, is: 0, ir: 0)
PASS (woot!):   Tier 30 :      Ran 1024x1024   image with 2594 bodies in 92ms (s: 27, r: 62, is: 0, ir: 0)
PASS (ayy!):    Tier 31 :      Ran 1072x1072   image with 2698 bodies in 96ms (s: 29, r: 64, is: 0, ir: 0)
PASS (eoo!):    Tier 32 :      Ran 1120x1120   image with 2806 bodies in 104ms (s: 31, r: 70, is: 0, ir: 0)
PASS (yeee!):   Tier 33 :      Ran 1152x1152   image with 2918 bodies in 111ms (s: 34, r: 74, is: 0, ir: 0)
PASS (eoo!):    Tier 34 :      Ran 1200x1200   image with 3035 bodies in 125ms (s: 36, r: 84, is: 0, ir: 2)
PASS (boyah!):  Tier 35 :      Ran 1248x1248   image with 3156 bodies in 129ms (s: 39, r: 84, is: 0, ir: 2)
PASS (eoo!):    Tier 36 :      Ran 1312x1312   image with 3283 bodies in 142ms (s: 42, r: 95, is: 0, ir: 3)
PASS (eoo!):    Tier 37 :      Ran 1360x1360   image with 3414 bodies in 155ms (s: 45, r: 105, is: 0, ir: 3)
PASS (yeee!):   Tier 38 :      Ran 1408x1408   image with 3551 bodies in 172ms (s: 48, r: 118, is: 0, ir: 3)
PASS (yay!):    Tier 39 :      Ran 1472x1472   image with 3693 bodies in 187ms (s: 54, r: 126, is: 0, ir: 3)
PASS (yeee!):   Tier 40 :      Ran 1536x1536   image with 3840 bodies in 197ms (s: 57, r: 133, is: 0, ir: 5)
PASS (woot!):   Tier 41 :      Ran 1584x1584   image with 3994 bodies in 215ms (s: 60, r: 146, is: 0, ir: 4)
PASS (woot!):   Tier 42 :      Ran 1648x1648   image with 4154 bodies in 225ms (s: 66, r: 157, is: 0, ir: 0)
PASS (boyah!):  Tier 43 :      Ran 1728x1728   image with 4320 bodies in 242ms (s: 69, r: 170, is: 0, ir: 0)
PASS (yeee!):   Tier 44 :      Ran 1792x1792   image with 4493 bodies in 256ms (s: 76, r: 178, is: 0, ir: 0)
PASS (skrrt!):  Tier 45 :      Ran 1856x1856   image with 4672 bodies in 281ms (s: 81, r: 195, is: 0, ir: 1)
PASS (yeee!):   Tier 46 :      Ran 1936x1936   image with 4859 bodies in 293ms (s: 87, r: 203, is: 0, ir: 0)
PASS (eoo!):    Tier 47 :      Ran 2016x2016   image with 5054 bodies in 315ms (s: 94, r: 218, is: 0, ir: 0)
PASS (yay!):    Tier 48 :      Ran 2096x2096   image with 5256 bodies in 351ms (s: 103, r: 245, is: 0, ir: 0)
PASS (yay!):    Tier 49 :      Ran 2176x2176   image with 5466 bodies in 376ms (s: 110, r: 263, is: 0, ir: 0)
PASS (yeee!):   Tier 50 :      Ran 2272x2272   image with 5685 bodies in 397ms (s: 121, r: 274, is: 0, ir: 0)
PASS (eoo!):    Tier 51 :      Ran 2352x2352   image with 5912 bodies in 424ms (s: 129, r: 293, is: 0, ir: 0)
PASS (yay!):    Tier 52 :      Ran 2448x2448   image with 6149 bodies in 447ms (s: 138, r: 305, is: 0, ir: 0)
PASS (ayy!):    Tier 53 :      Ran 2544x2544   image with 6395 bodies in 485ms (s: 151, r: 332, is: 0, ir: 0)
PASS (yeee!):   Tier 54 :      Ran 2656x2656   image with 6651 bodies in 526ms (s: 162, r: 361, is: 0, ir: 0)
PASS (ayy!):    Tier 55 :      Ran 2752x2752   image with 6917 bodies in 560ms (s: 176, r: 380, is: 0, ir: 0)
PASS (boyah!):  Tier 56 :      Ran 2864x2864   image with 7193 bodies in 590ms (s: 189, r: 398, is: 0, ir: 0)
PASS (woot!):   Tier 57 :      Ran 2992x2992   image with 7481 bodies in 632ms (s: 204, r: 425, is: 0, ir: 0)
PASS (yeee!):   Tier 58 :      Ran 3104x3104   image with 7780 bodies in 687ms (s: 221, r: 463, is: 0, ir: 0)
PASS (yay!):    Tier 59 :      Ran 3232x3232   image with 8092 bodies in 738ms (s: 238, r: 496, is: 0, ir: 0)
PASS (boyah!):  Tier 60 :      Ran 3360x3360   image with 8415 bodies in 772ms (s: 257, r: 512, is: 0, ir: 0)
PASS (eoo!):    Tier 61 :      Ran 3488x3488   image with 8752 bodies in 828ms (s: 280, r: 546, is: 0, ir: 0)
PASS (skrrt!):  Tier 62 :      Ran 3632x3632   image with 9102 bodies in 894ms (s: 301, r: 591, is: 0, ir: 0)
PASS (woot!):   Tier 63 :      Ran 3776x3776   image with 9466 bodies in 956ms (s: 324, r: 629, is: 0, ir: 0)
FAIL (timeout): Tier 64 :      Ran 3936x3936   image with 9845 bodies in 1041ms (s: 352, r: 686, is: 0, ir: 0) but the cutoff is 1000ms; used blowthrough 1
FAIL (timeout): Tier 65 :      Ran 4080x4080   image with 10238 bodies in 1112ms (s: 380, r: 730, is: 0, ir: 0) but the cutoff is 1000ms; used blowthrough 2
FAIL (timeout): Tier 66 :      Ran 4256x4256   image with 10648 bodies in 1175ms (s: 410, r: 764, is: 0, ir: 0) but the cutoff is 1000ms; used blowthrough 3
Reached tier 63 of 100
```